DAILY MULTIDISCIPLINARY BRIEF • CATEGORIA 1/6: AI & MATH

De vectores a la atencion

El recorrido completo en un solo documento: primero los cimientos, luego la maquinaria del deep learning

Fecha: 20 de agosto de 2026 • **Formato:** dos niveles + ejercicios por nivel • **Nivel:** fundamentos → intermedio

1 Como usar este documento

Este brief tiene **dos partes con niveles distintos**, pensadas para leerse en ese orden:

- **Parte I – Fundamentos.** Que es un vector, que mide una derivada y como esas dos ideas construyen una neurona. Sin demostraciones, con numeros pequenos que puedes verificar a mano. Si las matematicas estan oxidadas, empieza aqui.
- **Parte II – Intermedio.** Las mismas ideas llevadas a la maquinaria real: backpropagation, el optimizador Adam, normalizacion y residuales, y el mecanismo de atencion que sostiene a los modelos actuales.

Cada parte cierra con sus propios ejercicios y solucionario, asi que puedes detenerte al final de la Parte I y volver a la Parte II otro dia. Al final hay un glosario que cubre ambos niveles y las fuentes completas.

Mapa mental de todo el documento

$$\underbrace{x}_{\substack{\text{datos}\\ \text{(un vector)}}} \longrightarrow \underbrace{w \cdot x + b}_{\substack{\text{combinacion}\\ \text{(producto punto)}}} \longrightarrow \underbrace{\text{error}}_{\substack{\text{que tan mal}\\ \text{le fue}}} \longrightarrow \underbrace{\text{pendiente}}_{\substack{\text{derivada}\\ \text{del error}}} \longrightarrow \underbrace{\text{ajustar } w}_{\text{aprender}}$$

La Parte I explica cada caja de este diagrama por separado. La Parte II explica como se hace todo esto de forma eficiente y estable cuando hay millones de parametros y cientos de capas.

PARTE I • Fundamentos

Flechas que se suman y pendientes que se miden. Todo verificable a mano.

2 Los cimientos, uno por uno

1. Que es un vector (y por que se le llama flecha)

Sanderson, G. (3Blue1Brown)

La serie ensena algebra lineal empezando por la intuicion geometrica en vez de la notacion. Idea central: un vector es una flecha con direccion y longitud, y tambien es simplemente una lista de numeros. Las dos vistas son la misma cosa, y poder saltar entre ellas es lo que hace facil el resto.

Un vector en dos dimensiones es un par de numeros: $v = (3, 4)$. Puedes verlo como "camina 3 a la derecha y 4 hacia arriba". Solo puedes hacerle dos cosas basicas:

- **Sumarlo** con otro, componente por componente: $(3, 4) + (1, 2) = (4, 6)$.
- **Estirarlo** por un numero: $2 \cdot (3, 4) = (6, 8)$; la direccion no cambia, solo el tamano.

Su longitud (o *norma*) sale del teorema de Pitagoras: $\|v\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$.

Por que importa en IA

Cada dato que le das a un modelo es un vector. Una foto de 28×28 pixeles es un vector de 784 numeros. Una palabra se convierte en un vector de, digamos, 512 numeros. Todo lo que el modelo hace es mover estas flechas de lugar.

2. El producto punto: una medida de parecido

Deisenroth, Faisal & Ong (2020)

El libro de referencia para las matematicas justas y necesarias de ML. Sus capitulos 2 y 3 cubren vectores, matrices y geometria (productos internos, normas, proyecciones), que es exactamente el vocabulario que aparece en cada arquitectura moderna.

El **producto punto** toma dos vectores y devuelve un solo numero: multiplica componente por componente y suma.

$$w \cdot x = \sum_i w_i x_i, \qquad \text{ejemplo: } (2,1)\cdot(3,4) = 2\cdot 3 + 1\cdot 4 = 10.$$

Lo importante no es la formula, es lo que *mide*. Geometricamente, $w \cdot x = \|w\|\,\|x\|\cos\theta$, donde θ es el angulo entre las flechas. O sea:

- Si apuntan al mismo lado, el resultado es **positivo** (se parecen).
- Si son perpendiculares, es **cero** (no tienen nada que ver).
- Si apuntan en contra, es **negativo** (se oponen).

Una **matriz** es una tabla de numeros, y multiplicar matriz por vector es solo hacer varios productos punto a la vez: cada fila de la matriz da un numero.

$$\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}\begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 2\cdot 3 + 1\cdot 4 \\ 0\cdot 3 + 3\cdot 4 \end{pmatrix} = \begin{pmatrix} 10 \\ 12 \end{pmatrix}$$

3. La derivada: nada mas que una pendiente

Sanderson, G. (3Blue1Brown)

Presenta la derivada sin formalismo de limites: es la respuesta a "si muevo la entrada un poquito, cuanto se mueve la salida". La serie construye las reglas de derivacion desde la geometria, lo que las vuelve deducibles en lugar de memorizables.

Para $f(x) = x^2$, la derivada es $f'(x) = 2x$. Eso significa: en el punto $x = 3$, la curva sube con pendiente 6. Si te mueves un pelin a la derecha, la salida sube unas 6 veces ese pelin.

El **gradiente** es solo la version con varias variables: un vector que junta todas las pendientes, una por cada direccion. Y tiene una propiedad util: el gradiente apunta hacia *arriba*, hacia donde la funcion crece mas rapido. Asi que para *minimizar* algo, caminamos en la direccion contraria. Eso es el **descenso de gradiente**:

$$x_{\text{nuevo}} = x_{\text{viejo}} - \eta\, f'(x_{\text{viejo}}),$$

donde η (eta) es el *learning rate*: el tamano del paso.

La metafora de la colina

Imagina que estas en una montana con niebla y quieres bajar. No ves el valle, pero si sientes la inclinacion del suelo bajo tus pies. Das un paso hacia donde baja, vuelves a sentir, repites. Eso es entrenar un modelo. El learning rate es el tamano de tus pasos: muy chicos y tardas una eternidad; muy grandes y te pasas del valle de un brinco.

4. Sintesis: una neurona es las dos ideas juntas

Nielsen, M. (2015)

Construye redes neuronales desde cero con el problema de reconocer digitos escritos a mano. Empieza en la neurona individual y llega a backpropagation, introduciendo cada pieza matematica solo cuando se vuelve necesaria. Es el punto de entrada mas amable que existe al tema.

Una neurona hace exactamente dos cosas: un producto punto y una curva.

La neurona completa

$$y = \sigma(z), \qquad \text{donde} \qquad z = \underbrace{w \cdot x}_{\text{producto punto}} + \underbrace{b}_{\text{sesgo}}$$

x son los datos, w son los *pesos* (que tanto le importa cada entrada), b es el *sesgo* (que tan facil se activa) y σ es una curva que aplasta el resultado a un rango, por ejemplo entre 0 y 1.

Aprender significa buscar los w y b que hacen el error mas chico. Y ya sabemos como: calculamos la pendiente del error respecto a cada peso, y damos un paso en contra. Una red neuronal es esto mismo, apilado en capas.

Python • producto punto y un descenso de gradiente

```python
# 1) Producto punto a mano (sin librerias)
w = [2, 1]
x = [3, 4]
producto = sum(wi * xi for wi, xi in zip(w, x))   # 2*3 + 1*4 = 10

# 2) Minimizar f(x) = x**2 bajando por la pendiente
#    la derivada es f'(x) = 2x
x_actual = 4.0
lr = 0.1                          # learning rate: tamano del paso
for paso in range(5):
    pendiente = 2 * x_actual   # f'(x)
    x_actual = x_actual - lr * pendiente
    print(paso, round(x_actual, 4))
# 0 3.2   1 2.56   2 2.048   3 1.6384   4 1.31072
# se acerca a 0, que es donde x**2 es minimo
```

3 Ejercicios de la Parte I

Ejercicio 1. Vectores. Sea $v = (6, 8)$. (a) Calcula su longitud $\|v\|$. (b) Calcula $v + (1, -3)$. (c) Calcula $0.5 \cdot v$ y di si cambio su direccion.

Ejercicio 2. Producto punto y matrices. (a) Calcula $(1, 2) \cdot (4, 5)$. (b) Calcula $(1, 0) \cdot (0, 7)$ y explica en una linea que significa ese resultado. (c) Multiplica $\begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix} \begin{pmatrix} 2 \\ 5 \end{pmatrix}$.

Ejercicio 3. Pendiente y descenso. Sea $f(x) = x^2 - 4x$. (a) Su derivada es $f'(x) = 2x - 4$; calcula la pendiente en $x = 0$ y en $x = 3$. (b) Partiendo de $x = 0$ con learning rate $\eta = 0.1$, da *dos* pasos de descenso de gradiente. (c) Mirando a donde te vas moviendo, adivina en que valor de x esta el minimo.

▷ *Razonamiento.* Si el producto punto entre los pesos w de una neurona y un dato x sale **cero**, que quiere decir eso sobre la relacion entre ese dato y lo que la neurona aprendio a buscar? Y si sale muy negativo?

Solucionario – Parte I

E1. (a) $\|v\| = \sqrt{6^2 + 8^2} = \sqrt{36 + 64} = \sqrt{100} = 10$. (b) $(6 + 1,\ 8 - 3) = (7, 5)$. (c) $0.5 \cdot (6, 8) = (3, 4)$; la direccion *no* cambia, solo se hizo la mitad de largo (de hecho $(3, 4)$ apunta igual que $(6, 8)$).

E2. (a) $1 \cdot 4 + 2 \cdot 5 = 4 + 10 = 14$. (b) $1 \cdot 0 + 0 \cdot 7 = 0$; los vectores son perpendiculares, no comparten nada de direccion, asi que uno no aporta informacion sobre el otro. (c) Fila por fila: $1 \cdot 2 + 2 \cdot 5 = 12$ y $3 \cdot 2 + 0 \cdot 5 = 6$, o sea $\binom{12}{6}$.

E3. (a) $f'(0) = 2(0) - 4 = -4$ (baja hacia la derecha); $f'(3) = 2(3) - 4 = 2$ (sube hacia la derecha). (b) Paso 1: $x = 0 - 0.1(-4) = 0.4$. Paso 2: $f'(0.4) = 2(0.4) - 4 = -3.2$, entonces $x = 0.4 - 0.1(-3.2) = 0.72$. (c) Nos movemos hacia la derecha, acercandonos a donde la pendiente es cero: $2x - 4 = 0 \Rightarrow x = 2$. El minimo esta en $x = 2$.

Razonamiento. Si $w \cdot x = 0$, el dato es perpendicular a los pesos: no se parece nada al patron que la neurona busca, asi que no la activa ni a favor ni en contra (es informacion neutra para ella). Si sale muy negativo, el dato apunta en direccion *opuesta* al patron: es evidencia fuerte en contra, y con una curva σ tipo sigmoide la salida quedara cerca de 0.

PARTE II • Intermedio

Las mismas ideas, ahora a escala: como se entrena de verdad y por que funciona.

4 La maquinaria del deep learning

Hasta aqui tenemos una neurona y sabemos que aprender es bajar una colina. La Parte II responde tres preguntas que aparecen en cuanto la red crece: como se calculan las pendientes de *todas* las capas sin morir de costo computacional, como se dan pasos inteligentes en lugar de pasos fijos, y que trucos evitan que una red profunda se rompa al entrenar.

5. Backpropagation: la regla de la cadena a escala

Rumelhart, Hinton & Williams (1986)

Introduce el algoritmo que hace viable entrenar redes multicapa: propagar el error hacia atras aplicando la regla de la cadena capa por capa. Convierte el ajuste de millones de pesos en una serie de multiplicaciones matriz-vector, reutilizando los resultados del paso hacia adelante. Es la base de todo entrenamiento moderno.

En la Parte I calculamos la pendiente de una funcion de una variable. Con capas apiladas, la salida de una es la entrada de la siguiente, asi que la pendiente total es el *producto* de las pendientes intermedias: eso es la regla de la cadena. Para una capa $z^{(l)} = W^{(l)} a^{(l-1)}$:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^{\top}, \qquad \delta^{(l)} = \left(W^{(l+1)}\right)^{\top} \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right).$$

Lee la segunda ecuacion de derecha a izquierda: el error de la capa siguiente ($\delta^{(l+1)}$) se devuelve multiplicando por los pesos, y se filtra por la pendiente de la activacion. El costo total de backprop es del mismo orden que el paso hacia adelante, o sea $O(\#\text{parametros})$: ahi esta el truco, y por eso es viable.

6. Adam: pasos con memoria y escala propia

Kingma & Ba (2014)

Adam combina momento (una media movil del gradiente, que suaviza la direccion) con escalado adaptativo por coordenada (una media movil del gradiente al cuadrado, que ajusta el paso de cada parametro por separado). Corrige el sesgo inicial de ambas medias y produce un paso robusto que funciona bien con los hiperparametros por defecto, razon por la que domina el entrenamiento de redes grandes.

En la Parte I el paso era siempre η veces la pendiente. Adam mejora eso en dos sentidos: recuerda hacia donde venia bajando (momento) y le da pasos mas grandes a los parametros cuyo gradiente es consistentemente chico. Con $g_t = \nabla_\theta \mathcal{L}_t$:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$
$$\hat{m}_t = \tfrac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \tfrac{v_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta \, \tfrac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Python • un paso de Adam

```python
def adam_step(theta, g, m, v, t, lr=1e-3, b1=0.9, b2=0.999, eps=1e-8):
    m = b1 * m + (1 - b1) * g          # momento (1er momento)
    v = b2 * v + (1 - b2) * (g * g)    # escala adaptativa (2do momento)
    m_hat = m / (1 - b1 ** t)          # correccion de sesgo
    v_hat = v / (1 - b2 ** t)
    theta = theta - lr * m_hat / (v_hat ** 0.5 + eps)
    return theta, m, v
```

7. Estabilidad: normalizacion y conexiones residuales

Ba, Kiros & Hinton (2016) | He et al. (2015)

Dos trucos que permiten entrenar redes muy profundas. *Layer Norm* normaliza las activaciones por muestra (media y varianza sobre las features), estabilizando la escala de los gradientes. Las *conexiones residuales* $y = x + F(x)$ crean un camino directo para el gradiente, evitando que se desvanezca y permitiendo cientos de capas.

Layer Norm: $\hat{x}_i = \dfrac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$, con μ y σ^2 calculados sobre las features de *esa* muestra; luego se escala y desplaza con parametros aprendidos γ, β. El bloque residual convierte el objetivo de "aprender $H(x)$" en "aprender el residuo $F(x) = H(x) - x$", que suele ser mas facil de optimizar.

Por que los residuales salvan el gradiente

En $y = x + F(x)$, la derivada del termino x respecto a x es exactamente 1. Eso significa que hay un carril por el que el gradiente pasa *intacto* de la salida a la entrada, sin multiplicarse por numeros pequenos capa tras capa. Sin ese carril, multiplicar 200 numeros menores que 1 da practicamente cero: el gradiente se desvanece y las primeras capas dejan de aprender.

8. Atencion: el producto punto, otra vez

Vaswani et al. (2017)

El Transformer une todo lo anterior: es puro algebra lineal (productos $QK^\top$ y AV), se entrena con backprop y Adam, y depende de layer norm y residuales para ser estable. Reemplaza la recurrencia por una operacion de atencion global que conecta cualquier par de posiciones en un solo paso, lo que la vuelve paralelizable.

Aqui se cierra el circulo con la Parte I. La atencion pregunta, para cada posicion, "que tanto me importa cada una de las otras posiciones", y esa pregunta se responde con un **producto punto**: exactamente la medida de parecido de la seccion 2.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

$QK^\top$ da todos los parecidos entre posiciones; el softmax los convierte en pesos que suman 1; multiplicar por V produce un promedio ponderado. El escalado $1/\sqrt{d_k}$ mantiene la varianza de los numeros en ≈ 1 y evita que el softmax se sature (lo que aplanaria los gradientes).

5 Ejercicios de la Parte II

Ejercicio 4. **Escalado de la atencion.** Con q_i, k_i variables independientes de media 0 y varianza 1, demuestra que $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ tiene varianza d_k. Usa el resultado para justificar por que se divide por $\sqrt{d_k}$ y no por d_k.

Ejercicio 5. **Atencion a mano.** Calcula la atencion para $n = 2$, $d_k = 2$ con $Q = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$, $K = \left(\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right)$, $V = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 3 \end{smallmatrix}\right)$. Obten

(a) $S = QK^\top$, (b) $S/\sqrt{d_k}$, (c) $A = \mathrm{softmax}(S/\sqrt{d_k})$ por filas y (d) la salida AV.

Ejercicio 6. Correccion de sesgo en Adam. Con $\beta_1 = 0.9$, $\beta_2 = 0.999$ y gradiente constante $g_t = 1$, calcula $\hat{m}_1$ y $\hat{v}_1$ en el primer paso ($t = 1$) y explica por que la correccion es necesaria justo al inicio.

▷ *Razonamiento.* Un equipo entrena una red de 200 capas y el gradiente se desvanece. Como ayudan, por separado y en conjunto, las conexiones residuales y la normalizacion? Argumenta en terminos del flujo del gradiente.

Solucionario – Parte II

E4. Por independencia y media 0: $\mathrm{Var}(q \cdot k) = \sum_i \mathrm{Var}(q_i k_i) = \sum_i \mathrm{Var}(q_i)\mathrm{Var}(k_i) = \sum_i 1 \cdot 1 = d_k$. Luego la desviacion estandar es $\sqrt{d_k}$, y dividir por $\sqrt{d_k}$ renormaliza la varianza a 1. Dividir por d_k dejaria varianza $1/d_k$, aplanando en exceso el softmax y perdiendo capacidad de discriminar.

E5. (a) $S = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$. (b) $S/\sqrt{2} = \left(\begin{smallmatrix} 0.707 & 0.707 \\ 0 & 0.707 \end{smallmatrix}\right)$. (c) Fila 1: los dos logits son iguales $\Rightarrow A_1 = (0.5, 0.5)$. Fila 2: $\mathrm{softmax}(0,\ 0.707)$; $e^0 = 1$, $e^{0.707} = 2.028$, suma $3.028 \Rightarrow A_2 = (0.330, 0.670)$. (d) AV: fila 1 $= (0.5 \cdot 2,\ 0.5 \cdot 3) = (1.0,\ 1.5)$; fila 2 $= (0.330 \cdot 2,\ 0.670 \cdot 3) = (0.660,\ 2.010)$. Cada fila de la salida es el promedio de los valores ponderado por la atencion.

E6. $m_1 = (1 - 0.9) \cdot 1 = 0.1$, entonces $\hat{m}_1 = 0.1/(1 - 0.9) = 1.0$. $v_1 = (1 - 0.999) \cdot 1 = 0.001$, entonces $\hat{v}_1 = 0.001/(1 - 0.999) = 1.0$. Sin la correccion, $m_1 = 0.1$ y $v_1 = 0.001$ estarian fuertemente sesgados hacia 0 porque las medias se inicializan en cero; el cociente $m_1/\sqrt{v_1}$ daria un primer paso enorme y erratico. La correccion los lleva al valor insesgado ≈ 1.

Razonamiento. Los residuales $y = x + F(x)$ crean un atajo por el que el gradiente fluye sin atenuarse (la derivada del termino identidad es 1), evitando el desvanecimiento en profundidad. La normalizacion mantiene activaciones y gradientes en una escala estable capa a capa, evitando explosion o desvanecimiento por cambios de escala. Juntos permiten entrenar cientos de capas: los residuales garantizan el camino del gradiente, la normalizacion garantiza que su escala este controlada.

6 Glosario rapido (ambos niveles)

Termino	Que significa, en una linea
Parte I – Fundamentos	
Vector	Una lista de numeros; tambien una flecha con direccion y largo.
Norma $\|v\|$	El largo de la flecha, por Pitagoras.
Producto punto	Multiplicar componente por componente y sumar; mide cuanto se parecen dos vectores.
Matriz	Una tabla de numeros; multiplicarla por un vector es hacer varios productos punto de golpe.
Derivada	La pendiente de una curva en un punto: cuanto cambia la salida si muevo la entrada un poquito.
Gradiente	El vector con todas las pendientes; apunta hacia donde la funcion *sube* mas rapido.
Descenso de gradiente	Dar pasos en direccion contraria al gradiente para minimizar el error.
Learning rate (η)	El tamano de cada paso al bajar la colina.
Peso (w)	Que tanto le importa al modelo cada entrada; es lo que se ajusta al aprender.
Sesgo (b)	Un numero extra que corre el resultado; controla que tan facil se activa la neurona.
Activacion (σ)	La curva que aplasta el resultado a un rango (por ejemplo, entre 0 y 1).
Parte II – Intermedio	
Regla de la cadena	Para funciones encadenadas, la pendiente total es el producto de las pendientes.
Backpropagation	Aplicar la regla de la cadena hacia atras para obtener el gradiente de cada capa.
Funcion de perdida ($\mathcal{L}$)	El numero que mide que tan mal lo esta haciendo el modelo; es lo que se minimiza.
Momento	Media movil del gradiente; suaviza la direccion del paso.
Adam	Optimizador que combina momento con un paso adaptativo por parametro.
Layer Norm	Normalizar las activaciones de cada muestra para estabilizar escalas.
Conexion residual	El atajo $y = x + F(x)$ que deja pasar el gradiente intacto.
Softmax	Convierte una lista de numeros en pesos positivos que suman 1.
Atencion	Producto punto entre consultas y claves para ponderar valores.

7 Fuentes y citas

Parte I – Fundamentos

[1] Sanderson, G. (3Blue1Brown). *Essence of Linear Algebra* (serie visual). https://www.3blue1brown.com/lessons/eola-preview/

[2] Sanderson, G. (3Blue1Brown). *The Essence of Calculus* (serie visual). https://www.3blue1brown.com/lessons/essence-of-calculus/

[3] Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*, cap. 2–3. Cambridge University Press. https://mml-book.github.io

[4] Nielsen, M. (2015). *Neural Networks and Deep Learning*, cap. 1. http://neuralnetworksanddeeplearning.com

[5] Strang, G. *Linear Algebra* (18.06), MIT OpenCourseWare. https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

Parte II – Intermedio

[6] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature 323, 533–536. DOI: 10.1038/323533a0.

[7] Kingma, D. P., & Ba, J. (2014). *Adam: A Method for Stochastic Optimization*. arXiv:1412.6980. https://arxiv.org/abs/1412.6980

[8] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization*. arXiv:1607.06450. https://arxiv.org/abs/1607.06450

[9] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition*. arXiv:1512.03385. https://arxiv.org/abs/1512.03385

[10] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS. arXiv:1706.03762. https://arxiv.org/abs/1706.03762

Siguiente escalon

El puente que este documento deja pendiente es la *base ligera reforzada*: la regla de la cadena trabajada despacio con ejemplos de dos y tres capas, las funciones de activacion comparadas entre si (sigmoide, tanh, ReLU) y que es exactamente una funcion de perdida. Ese sera el mejor siguiente paso si la Parte I se siente comoda pero la Parte II todavia se siente rapida.